



DataGeo
Academy

CICLO DE FORMAÇÃO TÉCNICA

Sensoriamento Remoto e **Data Science**

Algoritmo para Extração de Feições de Imagens de Satélite de Baixa, Média e Alta Resolução Espacial

FLÁVIO ANTONIO OLIVEIRA DA SILVA

Engenheiro Cartógrafo



O que vamos explorar

Das etapas conceituais ao código Python completo para extração de feições

- | | | | |
|----|---|----|--|
| 01 | Introdução ao algoritmo e fluxo de trabalho | 05 | Código Python: bibliotecas e pré-processamento |
| 02 | Etapas: Pré-processamento e Realce de Feições | 06 | Código Python: realce, segmentação e extração |
| 03 | Etapas: Segmentação e Extração de Feições | 07 | Código Python: pós-processamento e execução |
| 04 | Etapas: Pós-processamento, Validação e Output | 08 | Ajustes, expansão e conclusão |

Extração de Feições: um fluxo adaptado à resolução

Desenvolver um algoritmo para extração de feições de imagens de satélite envolve várias etapas que variam dependendo da resolução espacial da imagem (baixa, média ou alta). O fluxo de trabalho geral é descrito abaixo, com as etapas principais para cada tipo de resolução.

Baixa Resolução

- Técnicas simples de thresholding
- Segmentação básica suficiente
- Processamento rápido e eficiente
- Plataformas: MODIS, Landsat (250m+)

Média Resolução

- Machine Learning supervisionado (SVM, RF)
- Segmentação por crescimento de regiões
- Fusão de bandas multiespectrais
- Plataformas: Sentinel-2 (10m), Landsat (30m)

Alta Resolução

- Deep Learning (CNNs) necessário
- Correção geométrica crucial e precisa
- Técnicas morfológicas refinadas
- Plataformas: Planet (3m), Pléiades (0.5m)

Pré-processamento e Realce de Feições

Preparação e melhoria da imagem antes da segmentação e extração



Pré-processamento

- Correção Radiométrica: ajuste de distorções e erros nos valores de pixel por variações de iluminação e ângulo de visão — essencial para todas as resoluções
- Correção Geométrica: alinhamento preciso da imagem à superfície terrestre. Crucial para imagens de alta resolução
- Subtração do Background: remoção de nuvens e ruídos usando máscaras de nuvens e técnicas de mascaramento



Realce de Feições

- Filtros de Realce: Sobel ou Laplaciano para destacar bordas; filtro de média para suavizar — enfatizam estruturas de interesse na imagem
- Transformadas: Transformada de Fourier ou Wavelet para realçar padrões específicos e extrair componentes de frequência relevantes
- Objetivo: amplificar as características visuais que diferenciam as feições antes da segmentação



Por que isso importa

- A qualidade dos dados de entrada impacta diretamente o desempenho de todo o algoritmo subsequente
- Artefatos não removidos nesta fase comprometem a segmentação e a classificação final
- Em imagens de alta resolução, erros geométricos geram deslocamentos de metros — imprecisões inaceitáveis no produto final



Segmentação e Extração de Feições

O coração do algoritmo: agrupar pixels e classificar objetos de interesse



Segmentação

- Thresholding: aplica limiares para separar feições do fundo. Simples e eficaz em imagens de baixa resolução
- Crescimento de Regiões: agrupa pixels conectados com características semelhantes. Ideal para média e alta resolução
- Clustering (K-Means, DBSCAN): agrupa pixels por características espectrais sem rótulos pré-definidos



Extração de Feições

- Classificação Supervisionada: SVM ou Random Forest para classificar pixels em classes de feições — mais eficaz em média/alta resolução
- Classificação Não Supervisionada: K-Means classifica pixels sem rótulos. Útil quando dados de treinamento são escassos
- Detecção de Bordas: Canny ou algoritmos de contorno para detectar bordas precisas das feições



Escolha pelo tipo de resolução

- Baixa resolução: Thresholding simples + K-Means não supervisionado são suficientes
- Média resolução: Crescimento de regiões + Random Forest supervisionado oferecem boa precisão
- Alta resolução: CNNs (Deep Learning) + detecção precisa de contornos são necessários para capturar detalhes finos

Pós-processamento, Validação e Output Final

Refinamento, avaliação de qualidade e entrega dos resultados geoespaciais



Pós-processamento

- Filtragem Morfológica: dilatação e erosão para refinar feições, remover ruídos e melhorar definição de bordas
- Fusão de Dados: combinação de diferentes bandas espectrais ou imagens de resoluções distintas para melhorar precisão
- Objetivo: eliminar artefatos residuais e tornar as feições extraídas mais precisas e geometricamente consistentes



Validação e Ajustes

- Validação Cruzada: comparação dos resultados com dados de referência (ground truth) para avaliar a acurácia do algoritmo
- Ajustes Finais: modificação dos parâmetros com base nos resultados da validação para melhorar a precisão global
- Métricas: acurácia geral, coeficiente Kappa, matriz de confusão e F1-Score por classe de feição



Output Final

- Geração de Mapas Temáticos: mapas que destacam as feições extraídas, prontos para uso em SIG
- Análise Estatística: cálculo de métricas quantitativas — área, perímetro, densidade, distribuição espacial
- Exportação em formatos geoespaciais padrão: GeoTIFF, Shapefile, GeoJSON para integração em fluxos de trabalho SIG

Pipeline completo: da imagem bruta ao mapa temático

Sete etapas seqüenciais que transformam imagens de satélite em feições geoespaciais





Bibliotecas e Pré-processamento

NumPy, OpenCV e scikit-learn — instalação, importação e função de pré-processamento

Python — Extração de Feições em Imagens de Satélite

```
1  # Instalação das bibliotecas
2  # pip install numpy opencv-python scikit-learn
3
4  # Importação das bibliotecas
5  import numpy as np
6  import cv2
7  from sklearn.cluster import KMeans
8  from sklearn.ensemble import RandomForestClassifier
9  from skimage import morphology
10
11 # Função de Pré-processamento
12 def preprocess_image(img):
13     # Correção radiométrica (normalização)
14     img = cv2.normalize(img, None, 0, 255, cv2.NORM_MINMAX)
15     # Correção geométrica (placeholder – normalmente feita externamente)
16     # Subtração do background (por exemplo, remoção de nuvens)
17     img = cv2.GaussianBlur(img, (5, 5), 0)
18     # Verifica se a imagem já foi pré-processada para remover nuvens
19     return img
```



Bibliotecas: NumPy (operações matriciais), OpenCV (processamento de imagens), scikit-learn (ML), scikit-image (morfologia). Instale com: `pip install numpy opencv-python scikit-learn scikit-image`



Realce, Segmentação e Extração de Feições

Filtro Sobel, thresholding + K-Means e classificação com Random Forest

Python — Extração de Feições em Imagens de Satélite

```
1  def enhance_features(img):
2      # Aplicar filtro Sobel para realce de bordas
3      sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
4      sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
5      sobel = np.hypot(sobelx, sobely)
6      return sobel
7
8  def segment_image(img):
9      # Thresholding simples
10     _, thresh = cv2.threshold(img, 128, 255, cv2.THRESH_BINARY)
11     # Segmentação por K-Means
12     kmeans = KMeans(n_clusters=3, random_state=0)
13     reshaped = img.reshape(-1, 1).astype(float)
14     labels = kmeans.fit_predict(reshaped)
15     return labels.reshape(img.shape)
16
17  def extract_features(img, labels):
18     # Classificação supervisionada (Random Forest)
19     X = img.reshape(-1, 1); y = labels.flatten()
20     clf = RandomForestClassifier(n_estimators=100)
21     clf.fit(X, y)
22     return clf.predict(X).reshape(img.shape)
```



Realce com Sobel (magnitude do gradiente). Segmentação com K-Means (k=3 classes). Extração com Random Forest supervisionado — em produção, y deve vir de ground truth rotulado.



Pós-processamento e Exemplo de Execução

Filtragem morfológica, validação com ground truth e pipeline completo

```
Python — Extração de Feições em Imagens de Satélite

1  def postprocess(img):
2      # Filtragem morfológica: erosão e dilatação
3      struct = morphology.disk(3)
4      img = morphology.erosion(img, struct)
5      img = morphology.dilation(img, struct)
6      return img
7
8  def validate(extracted, ground_truth):
9      # Comparar features extraídas com dados de referência
10     accuracy = np.mean(extracted == ground_truth)
11     print(f'Acurácia: {accuracy:.2%}')
12     return accuracy
13
14 # — EXEMPLO DE EXECUÇÃO —
15 img = cv2.imread('satellite_image.tif', cv2.IMREAD_GRAYSCALE)
16 img = preprocess_image(img)      # Pré-processamento
17 enhanced = enhance_features(img) # Realce de feições
18 segmented = segment_image(img)   # Segmentação
19 extracted = extract_features(img, segmented) # Extração
20 result = postprocess(extracted.astype(np.uint8)) # Pós-proc
21 cv2.imwrite('extracted_features.tif', result * 50)
```



Pipeline completo em 5 chamadas. Substitua 'satellite_image.tif' pela imagem real. O fator *50 na escrita é para visualização — ajuste conforme a escala de classes do seu produto final.

Adaptável a qualquer resolução e contexto

O algoritmo é uma base inicial — cada aplicação exige ajuste fino dos parâmetros e métodos



Alta Resolução

- Ajustar parâmetros de segmentação para capturar detalhes finos (bordas menores)
- Usar Random Forest ou CNN em vez de thresholding simples para maior precisão
- Correção geométrica rigorosa com GCPs (pontos de controle terrestres)
- Aumentar número de clusters no K-Means para distinguir sub-classes de feições



Baixa Resolução

- Simplificar o algoritmo removendo etapas como filtragem morfológica fina
- Thresholding simples + K-Means com poucos clusters é suficiente e eficiente
- Foco na análise de áreas grandes: desmata-mento, águas, zonas agrícolas
- Processamento mais rápido permite análise de grandes extensões geográficas



Expansões Possíveis

- Integrar mais dados: altitude, tipos de solo, dados climáticos como features adicionais
- Testar Deep Learning (U-Net, ResNet) para segmentação semântica de alta precisão
- Implementar em Google Earth Engine ou AWS para escala global e tempo real
- Adicionar pipeline de atualização automática com novos dados de satélite

Um algoritmo flexível para qualquer escala de análise

Resumo: O algoritmo desenvolvido oferece uma base sólida e adaptável para a extração de feições de imagens de satélite em qualquer resolução. Cada etapa — pré-processamento, realce, segmentação, extração, pós-processamento e validação — pode ser ajustada conforme o tipo de imagem e a feição de interesse.

Perspectiva Futura: A integração com Deep Learning (U-Net, SAM — Segment Anything Model) e plataformas de nuvem como Google Earth Engine ou AWS SageMaker representará o próximo passo para tornar o algoritmo escalável e automático em produção global.



FLEXIBILIDADE

Adaptável da baixa à alta resolução

Da correção radiométrica ao mapa temático final — um pipeline Python completo e expansível.



DataGeo
Academy

Obrigado!

Conhecimento que gera decisões — do pixel ao mapa temático.

FLÁVIO ANTONIO OLIVEIRA DA SILVA • Engenheiro Cartógrafo